Problem A. 百万富翁

Time Limit 6000 ms
Mem Limit 524288 kB

Background

本题仅支持 C++ 语言评测。由于平台限制，使用 C++14 (GCC 9) 提交代码会导致 CE。请使用其他版本的 C++ 提交（推荐 C++14 及以上）。

与 NOI 要求的提交格式不同，你的程序中**不应该**包含头文件 `richest.h`。同时，你的程序中应当在包含 `vector` 头文件的前提下，包含对以下函数的声明：

```
std::vector<int> ask(std::vector<int> a, std::vector<int> b);
```

除此之外，其余要求与 NOI 要求一致。

Description

小 Y 的银行有 N 个客户，编号为 0 到 $N-1$。客户 i 有 W_i 元存款，且**客户之间的存款金额互不相同**。

小 P 是小 Y 的深度合作伙伴，他希望知道哪个客户的存款最多。小 P 无法直接获取客户的存款金额，但他可以依次发送若干次**请求**，每次请求包含若干个**查询**，每个查询是一个二元组 (i, j)，表示小 P 想知道客户 i 和客户 j 的存款金额哪个更多。如果 $W_i > W_j$，小 Y 会回答 i，否则回答 j。

小 P 的**请求数** t 和所有请求的**查询**次数总和 s 有上限，他希望你帮他写一个程序，来找到存款最多的客户。

Input

选手不需要，也不应该实现 main 函数。

选手应确保提交的程序包含头文件 `richest.h`，可在程序开头加入以下代码实现：

```
#include "richest.h"
```

选手需要实现以下函数：

```
int richest(int N,int T,int S);
```

- N 表示客户的数量；
- T 表示对于当前函数调用，**请求**数 t 不应超过此值；
- S 表示对于当前函数调用，所有请求的**查询**次数总和 s 不应超过此值；
- 该函数需要返回存款最多的客户的编号；
- 对于每个测试点，该函数**会被交互库调用恰好 10 次**；

选手可以通过调用以下函数向交互库发送一次**请求**：

```
std::vector <int> ask(std::vector <int> a, std::vector <int> b);
```

- 在调用 `ask` 函数时需要保证传入参数 a 和 b 的长度相同，且其中的每个元素都必须是小于 N 的非负整数，表示该**请求**中的所有**查询**。
- 该函数会返回一个类型为 `std::vector <int>` 且长度与 a 和 b 相同的变量，设为 c，其中 $c[i]$ 表示在客户 $a[i]$ 和 $b[i]$ 中存款较多的客户的编号。

题目保证在规定的**请求**和**查询**次数限制下，交互库运行的时间不超过 3 秒，交互库使用的内存大小固定，且不超过 256 MiB。

试题目录下的 `grader.cpp` 是提供的交互库参考实现，最终测试所用的交互库与该参考实现有所不同，因此选手的解法不应该依赖交互库实现。

选手可以在本题目录下使用如下命令编译得到可执行程序：

```
g++ grader.cpp richest.cpp -o richest -O2 -std=c++14 -static
```

对于编译得到的可执行程序：

- 可执行文件将从**标准输入**读入以下格式的数据：
  - 输入的第一行包含四个非负整数 N, T, S, R，其中 R 是交互库生成测试数据的随机种子。
- 输入完成后，交互库将调用 10 次函数 `richest`，用输入的参数生成的测试数据进行测试。`richest` 函数返回后，交互库会输出以下信息：
  - 输出的前 10 行中，每行首先包含三个整数 r, t, s，表示该次执行的结果。其中 r 是 `richest` 函数的返回值，t 和 s 的含义如题目描述中所示，然后包含该次运行的正确性等消息。
  - 输出的第 11 行包含 10 次运行的总信息。

Output

假设可执行文件生成的测试数据为 $N = 4$，$W = [101, 103, 102, 100]$，$T = 100$，$S = 100$，下面是一个正确的交互过程：

::cute-table{tuack}

选手程序	交互库	说明
	调用 `richest(4,100,100)`	开始测试
调用 `ask([0,2],[1,3])`	返回 $[1, 2]$	$W_0 < W_1, W_2 > W_3$
调用 `ask([0,2,3],[1,1,1])`	返回 $[1, 1, 1]$	$W_0 < W_1, W_2 < W_1, W_3 < W_1$
运行结束并返回 1	向屏幕打印交互结果	交互结束，结果正确

在这个例子中，$r = 1, t = 2, s = 5$ 满足请求与查询次数的限制。

Hint

【下发文件说明】

在本试题目录下：

- `grader.cpp` 是提供的交互库参考实现。
- `richest.h` 是头文件，选手不用关心具体内容。
- `template_richest.cpp` 是提供的示例代码，选手可在此代码的基础上实现。

选手注意对所有下发文件做好备份，最终评测时只测试本试题目录下的 `richest.cpp`，对该程序以外文件的修改不会影响评测结果。

【数据范围】

对于所有测试数据保证：所有 W_i 两两不同。

本题共 2 个测试点，每个测试点的分值和数据范围见下表。

::cute-table{tuack}

测试点编号	分值	$N =$	$T =$	$S =$
1	15	$1\,000$	1	$499\,500$
2	85	$1\,000\,000$	20	$2\,000\,000$

【评分方式】

注意：

- 选手不应通过非法方式获取交互库的内部信息，如试图直接读取数组 W 的值，或直接与标准输入、输出流进行交互。此类行为将被视为作弊。
- **最终的评测交互库与样例交互库的实现不同，且可能是适应性的：在不与 `ask` 此前返回的结果相矛盾的前提下，最终的评测交互库可能会动态调整 W 的值。**

本题首先会受到和传统相同的限制。例如编译错误会导致整道题目得 0 分，运行时错误、超过时间限制、超过空间限制都会导致相应测试点得 0 分。选手只能在程序中访问自己定义的和交互库给出的变量或数据，及其相应的内存空间。尝试访问其他位置空间将可能导致编译错误或运行错误。

在每次 `richest` 函数调用中，程序使用的请求次数 t 和所有请求的查询次数总和 s 需在对应限制下，否则将会获得 0 分。

在上述条件基础上：

- 在测试点 1 中，程序得到满分当且仅当 `ask` 函数调用合法且 `richest` 函数返回的答案正确；
- 在测试点 2 中，程序得到的分数将按照以下方式计算：
  - 若 `ask` 函数调用不合法，则获得 0 分；
  - 若 `ask` 函数调用均合法，设 $\max t$ 表示多次调用 `richest` 函数所得的 t 的最大值，$\max s$ 表示 s 的最大值，则程序将获得 $\lfloor 85 \cdot f(\max t) \cdot g(\max s) \rfloor$ 分，其中 f 与 g 的计算方式如下表所示：

::cute-table{tuack}

$\max t$	$f(\max t)$
$\max t \le 8$	1
$9 \le \max t \le 20$	$1 - \frac{1}{4}\sqrt{\max t - 8}$

::cute-table{tuack}

$\max s$	$g(\max s)$
$\max s \le 1\,099\,944$	1
$1\,099\,945 \le \max s \le 1\,100\,043$	$1 - \frac{1}{6}\log_{10}(\max s - 1\,099\,943)$
$1\,100\,044 \le \max s \le 2\,000\,000$	$\frac{2}{3} - \frac{1}{1\,500}\sqrt{\max s - 1\,100\,043}$

以下是测试点 2 中，不同的 t 和 s 对得分影响的示例。

::cute-table{tuack}

$\max t$	$\max s$	测试点 2 的得分
$= 20$	$\leq 1\,099\,944$	11
$= 19$	$\leq 1\,099\,944$	14
$= 18$	$\leq 1\,099\,944$	17
$= 17$	$\leq 1\,099\,944$	21
$= 16$	$\leq 1\,099\,944$	24
$= 15$	$\leq 1\,099\,944$	28
$= 14$	$\leq 1\,099\,944$	32
$= 13$	$\leq 1\,099\,944$	37
$= 12$	$\leq 1\,099\,944$	42
$= 11$	$\leq 1\,099\,944$	48
$= 10$	$\leq 1\,099\,944$	54
$= 9$	$\leq 1\,099\,944$	63
≤ 8	$\in [1\,099\,974, 1\,099\,978]$	63
≤ 8	$\in [1\,099\,969, 1\,099\,973]$	64
≤ 8	$\in [1\,099\,965, 1\,099\,968]$	65
≤ 8	$\in [1\,099\,962, 1\,099\,964]$	66
≤ 8	$\in [1\,099\,959, 1\,099\,961]$	67
≤ 8	$\in [1\,099\,957, 1\,099\,958]$	68
≤ 8	$\in [1\,099\,955, 1\,099\,956]$	69
≤ 8	$\in [1\,099\,953, 1\,099\,954]$	70
≤ 8	$= 1\,099\,952$	71
≤ 8	$= 1\,099\,951$	72
≤ 8	$\in [1\,099\,949, 1\,099\,950]$	73
≤ 8	$= 1\,099\,948$	75

$\max t$	$\max s$	测试点 2 的得分
≤ 8	$= 1\,099\,947$	76
≤ 8	$= 1\,099\,946$	78
≤ 8	$= 1\,099\,945$	80
≤ 8	$\leq 1\,099\,944$	85

Problem B. 元旦激光炮

Time Limit 1000 ms
Mem Limit 262144 kB

刚刚越过绝境长城，只见天空中出现了炫目的光芒 —— 圣诞老人出现了。

元旦三侠立刻进入战斗。生蛋侠、圆蛋侠和零蛋侠分别有 n_a, n_b, n_c 个激光炮。生蛋侠的激光炮的威力分别为 $a_0, a_1, \ldots, a_{n_a-1}$，圆蛋侠的激光炮的威力分别为 $b_0, b_1, \ldots, b_{n_b-1}$，零蛋侠的激光炮的威力分别为 $c_0, c_1, \ldots, c_{n_c-1}$。

元旦三侠的激光炮的威力**已经按从小到大的顺序排好序了**，即 $a_{i-1} \leq a_i$，$b_{i-1} \leq b_i$，$c_{i-1} \leq c_i$。

由于元旦三侠精力有限，他们得废弃掉 k 个激光炮。为了更好地进行战斗，他们决定废弃掉威力前 k 小的激光炮。

赶快帮助元旦三侠，让激光炮投入战斗吧！你只需要告诉他们威力第 k 小的激光炮威力是多少。

任务

你需要编写一个函数 query_kth，以确定威力值第 k 小的激光炮威力值是多少。

- query_kth(n_a, n_b, n_c, k)
  - n_a：生蛋侠拥有的激光炮数目 n_a。保证 $n_a \geq 0$。
  - n_b：圆蛋侠拥有的激光炮数目 n_b。保证 $n_b \geq 0$。
  - n_c：零蛋侠拥有的激光炮数目 n_c。保证 $n_c \geq 0$。
  - k：要查询的排名 k。保证 $1 \leq k \leq n_a + n_b + n_c$。

你可以调用三个函数 get_a、get_b、get_c 以帮助你确定第 k 小的激光炮。我们会根据你调用这三个函数的**总次数**评分。

- get_a(i) 将返回 a_i。如果 i 在 $0 \leq i < n_a$ 之外，该函数将返回 2147483647。
- get_b(i) 将返回 b_i。如果 i 在 $0 \leq i < n_b$ 之外，该函数将返回 2147483647。
- get_c(i) 将返回 c_i。如果 i 在 $0 \leq i < n_c$ 之外，该函数将返回 2147483647。

在一组测试数据中，query_kth 只会被调用一次。

实现细节

本题只支持 C/C++/Pascal。

你只能提交一个源文件实现如上所述的 query_kth 函数，并且遵循下面的命名和接口。

C/C++

你需要包含头文件 kth.h。

```
int query_kth(int n_a, int n_b, int n_c, int k);
```

函数 get_a, get_b, get_c 的接口信息如下。

```
int get_a(int p);
int get_b(int p);
int get_c(int p);
```

Pascal

你需要使用单元 graderhelperlib。

```
function query_kth(n_a, n_b, n_c, k : longint) : longint;
```

函数 get_a, get_b, get_c 的接口信息如下。

```
function get_a(p : longint) : longint;
function get_b(p : longint) : longint;
function get_c(p : longint) : longint;
```

如果有不清楚的地方，见样例及测评库下载，**内附了样例程序**。

评测方式

评测系统将读入如下格式的输入数据：

1. 第 1 行: n_a, n_b, n_c, k
2. 第 2 行：n_a 个整数，第 i 个整数表示 a_i。
3. 第 3 行：n_b 个整数，第 i 个整数表示 b_i。
4. 第 4 行：n_c 个整数，第 i 个整数表示 c_i。

在 query_k 返回后，评测系统将输出你的答案以及 get_a, get_b, get_c 三个函数的总调用次数。

样例一

input

```
2 3 3 5
1 2
```

```
1 5 6
2 3 3
```

output

```
3 6
```

explanation

所有激光炮从小到大排序后为 $1, 1, 2, 2, 3, 3, 5, 6$，所以第 5 小的数为 3。输出的第二个整数 6 为总调用次数，你可以认为这是一个调用了 6 次 get_a, get_b, get_c 函数的程序的输出。

样例二

见样例及测评库下载。

限制与约定

共 10 个测试点，每个测试点 10 分。设你的程序 get_a, get_b, get_c 函数的调用次数为 t。当 $t \leq 100$ 时得 10 分，否则当 $t \leq 2000$ 时得 6 分，否则不得分。

测试点编号	特殊限制
1	$n_a, n_b, n_c \leq 30$
2	$n_c = 0$
3	
4	
5	无
6	
7	
8	
9	
10	

对于所有测试点，$0 \leq n_a, n_b, n_c \leq 10^5$，$1 \leq a_i, b_i, c_i \leq 10^9$。

交互式类型的题目怎么本地测试

时间限制：1s

空间限制：256MB

下载

样例及测评库下载

Problem C. 生产计划 (Day 2)

Time Limit 3000 ms
Mem Limit 524288 kB

Background

翻译自 ROI 2023 D2T4。

这是一道 IO 交互题。

某个国家准备制定汽车的生产计划。

该国总共有 n 个城市，每个城市中有一个工厂，它们要参与汽车的生产。第 i 个城市的工厂可以雇佣 l_i 到 r_i 个人。

一些城市之间通过双向道路相连，且道路网络呈树状结构。因此，如果要从一个城市到另一个城市，在不重复经过同一个城市的情况下，只有一种路径。

生产计划被定义为一个整数序列 $[a_1, a_2, \ldots, a_n]$，其中 a_i 表示在第 i 个工厂工作的人数，满足 $l_i \leq a_i \leq r_i$。在制定生产计划后，一些工厂将被选为装配工厂，这些工厂负责装配汽车，而其它工厂则负责生产零部件。如果没有两个装配工厂所在的城市被一条道路直接连接，那么这个生产计划就是合理的。在所有可能的合理的计划中，会选择装配工厂的工人总数最大的一个计划。这个数就被称为计划 $[a_1, a_2, \ldots, a_n]$ 的效率。

Description

你需要确定是否存在效率为 v_i 的计划。如果存在这样的计划，则可能需要输出这样一个计划。

制定计划的过程包括两个阶段。

在第一阶段，你将获得 $v_1, v_2, \ldots, v_q$ 的值。你需要确定是否存在效率为 v_i 的计划，如果不存在，输出 `-1`；如果存在，输出一个非负整数 k_i。

对 k 的定义如下：给定三个整数 x, y, m，定义计划 $[a_1, a_2, \ldots, a_n]$ 的 $k = \bigoplus_{j=1}^{n} ((x \times j + y \times a_j^2) \bmod m)$，其中 $\oplus$ 为按位异或。

因为符合要求的计划可能不止一个，所以 k_i 也有很多种可能，你需要输出其中一种可能的 k_i。

在第二阶段，某些计划的可行性将被检查。在每个查询中，你的程序会得到一个整数 i（$1 \leq i \leq q$）。对于这个查询，如果不能制定效率为 v_i 的计划，你需要输出 `-1`；否则，你需要输出 n 个整数

$a_1, a_2, \ldots, a_n$（$l_i \le a_i \le r_i$），表示一个制定的计划。这个计划计算出来的 k 值应该等于你先前输出的 k_i，效率应该等于 v_i。

在程序输出每个 k 之前，无法知道哪些计划会被检查。因此，你需要保证在第一阶段中输出的 k_i 是正确的。

Input

本题多测，第一行输入一个整数 t（$1 \le t \le 10^4$），表示数据组数。

在每组数据中：

第一行包含两个整数 n, q（$2 \le n \le 2 \times 10^5$，$1 \le q \le 2 \times 10^5$），分别表示城市的数量和要制定的计划的数量。

第二行包含三个整数 x, y, m（$11 \le m \le 2^{30}$，$0 \le x, y < m$），即计算 k 的参数。

接下来的 $n-1$ 行描述了城市之间的道路网络。在这 $n-1$ 行中，第 i 行包含两个整数 s_i, f_i（$1 \le s_i, f_i \le n$，$s_i \neq f_i$），表示第 i 条道路连接城市 s_i 和 f_i 的双向道路。保证这些道路能够构成一棵树。

接下来的 n 行中的第 i 行包含两个整数 l_i, r_i（$0 \le l_i \le r_i \le 10^9$），表示第 i 个城市的工人数量限制。

接下来的一行包含 q 个整数 $v_1, v_2, \ldots, v_q$（$0 \le v_i \le \sum_{i=1}^{n} r_i$），表示需要制定的计划的效率值。保证所有的 v_i 都是不同的。

接下来，你需要输出第一部分中你需要输出的内容（具体方式见“输出格式”）。输出完成后，才可以读入第二部分的数据。

接下来若干行，每行输入一个数 i（$0 \le i \le q$）。当 $i \neq 0$ 时，i 表示要检查的计划的编号。在按照输出格式的要求输出后，交互库才会继续给你下一个 i。当 $i = 0$ 时，表示第二部分的检查结束，即这一组数据输入完成。如果这不是最后一组数据，你的程序应该立即开始读入下一组数据。

保证 $\sum n, \sum q \le 2 \times 10^5$，且每组数据中检查计划次数 c 的总和不超过 10^4，$\sum n \times c \le 10^6$。

Output

读取完每组输入的第一部分后，需要输出 q 个整数 $k_1, k_2, \ldots, k_q$（$-1 \le k_i < 2^{30}$），如果 $k_i = -1$ 则表示无法制定效率值为 v_i 的计划。输出后，交互库才会给你第二部分的数据。

在读入第二部分检查的计划编号 i（且 $i \neq 0$）后，如果第 i 个计划无法制定，即 $k_i = -1$ 时，则输出 `-1`。否则，需要输出 n 个整数 $a_1, a_2, \ldots, a_n$（$l_i \leq a_i \leq r_i$），表示所制定的计划。该计划的 k 应等于 k_i，并且效率应等于 v_i。

IO 交互题在输出后应刷新缓冲区。

如果在 C++ 中使用了 `cout << ... << endl`，在 Java 中使用了 `System.out.println`，在 Python 中使用了 `print`，在 Pascal 中使用了 `writeln`，那么输出缓冲区会自动刷新，不需要额外操作。

如果使用了其他输出方法，建议手动刷新输出缓冲区。在刷新输出缓冲区时，可以使用 C 和 C++ 中的 `fflush(stdout)`，Pascal 中的 `flush(output)`，Java 中的 `System.out.flush()`，Python 中的 `sys.stdout.flush()`。

Sample 1

Input	Output
1 9 3 4 7 15 1 2 2 4 2 5 1 3 3 6 3 7 6 8 6 9 4 4 2 2 5 5 3 3 2 2 6 6 3 3 4 4 3 3 18 19 20 1 2 3 0	-1 10 -1 -1 4 2 5 3 2 6 3 4 3 -1

Sample 2

Input	Output
3 3 4 3 4 11 1 2 2 3 0 1 0 1 0 1 0 1 2 3 2 0 4 6 1 2 11 1 2 2 3 3 4 0 2 1 1 1 1 1 2 0 1 2 3 4 5 5 2 3 0 5 7 11 31 101 1 2 2 3 2 4 3 5 1 2 1 5 0 4 1 4 4 6 13 12 11 10 9 8 6 3 5 7 0	12 8 3 -1 1 0 0 -1 -1 4 14 9 -1 2 1 1 2 -1 1 1 1 1 -1 127 23 58 13 90 91 2 5 4 1 6 2 4 4 3 4 1 1 0 1 4

Hint

在样例中，程序与交互库的交互用空行分隔。这样做是为了便于理解，清楚地知道哪个消息是对哪个消息的回应。在实际测试中，不会有多余的空行（当然，会有换行）。

第一个样例只有一种制定计划的方法，即 $a = [4, 2, 5, 3, 2, 6, 3, 4, 3]$。它的效率为 19，计算出来的 k 为 10。

第二个样例有三组数据。

- 在第一组数据中：
  - 存在唯一一个效率为 0 的计划，$a = [0, 0, 0]$。该计划的 k 为 12。
  - 存在效率为 1 的计划，例如 $a = [1, 0, 0]$。该计划的 k 为 8。
  - 存在效率为 2 的计划，例如 $a = [1, 0, 1]$。该计划的 k 为 3。
  - 不存在效率为 3 的计划。
  - 在这四个请求的计划中，检查了计划编号 $i = 2$（$v_2 = 1$）。
- 在第二组数据中：
  - 不存在效率为 0 的计划。
  - 不存在效率为 1 的计划。
  - 存在效率为 2 的计划，例如 $a = [1, 1, 1, 1]$。该计划的 k 为 4。
  - 存在效率为 3 的计划，例如 $a = [2, 1, 1, 1]$。该计划的 k 为 14。
  - 存在唯一的效率为 4 的计划 $a = [2, 1, 1, 2]$。该计划的 k 为 9。
  - 不存在效率为 5 的计划。
  - 在这六个请求的计划中，检查了计划编号 $i = 5$（$v_5 = 4$），$i = 2$（$v_2 = 1$）和 $i = 3$（$v_3 = 2$）。
- 在第三组数据的第一个部分中，请求了以下计划（从第二个开始，因为效率为 $v_1 = 13$ 的计划不存在）：$[2, 5, 4, 4, 6]$；$[2, 5, 4, 1, 6]$；$[2, 4, 4, 4, 4]$；$[2, 4, 4, 3, 4]$；$[2, 4, 4, 2, 4]$；$[1, 1, 0, 1, 4]$。

子任务	分值	$n, \sum n$	$\sum q$	其它特殊性质
1	11			$l_i = r_i$
2	9			$c = 0$
3	12			$l_i = 0, r_i \le 1$
4	4			$l_i = 0$
5	8			$s_i = 1, f_i = i + 1$
6	5	$n \le 10, \sum n \le 1000$	$Q \le 1000$	$c = q, r_i \le 10, r_i - l_i \le 2$
7	2	$n \le 10, \sum n \le 1000$	$Q \le 1000$	$c = q, r_i \le 10$
8	13	$\sum n \le 1000$	$Q \le 1000$	$c = q, \sum\limits_{i=1}^{n} r_i - l_i \le 10^4$
9	11	$\sum n \le 1000$	$Q \le 1000$	$c = q$

子任务	分值	$n, \sum n$	$\sum q$	其它特殊性质
10	6			$c = q$
11	5	$\sum n \le 1000$		
12	5	$\sum n \le 8000$		
13	9			

保证 $\sum n, \sum q \le 2 \times 10^5$，且每组数据中检查计划次数 c 的总和不超过 10^4，$\sum n \times c \le 10^6$。

Problem D. 末日魔法少女计划

Time Limit 2000 ms
Mem Limit 524288 kB

Background

CTT2021 D1T1

Description

对于给定的 n, k，你需要构造一个只含 $0, 1$ 的矩阵 $A_{i,j}$，$0 \le i, j \le n$，满足：

1. $A_{i,i} = 1$；
2. $A_{i,i+1} = 1$；
3. 对 $i > j$ 有 $A_{i,j} = 0$；
4. 若 $A_{i,j} = 1$，$j - i > 1$，则存在 $i < t < j$，满足 $A_{i,t} = A_{t,j} = 1$；
5. 对 $i \le j$ 有 $(A^k)_{i,j} > 0$。

你需要输出满足 $A_{i,j} = 1$ 且 $j - i > 1$ 的每个 (i, j)，设这样的 (i, j) 共有 m 个。

若输出不满足要求，则不能得到该测试点的任何分数。若输出满足要求，则根据 m 进行评分。

Input

一行，两个整数 n, k。

Output

第一行一个整数 m，接下来 m 行，每行两个整数 i, j，依次表示每个满足 $A_{i,j} = 1$ 且 $j - i > 1$ 的二元组 (i, j)。

Sample 1

Input	Output
3 2	1 0 2

Hint

- $1900 \leq n \leq 2000$;
- $2 \leq k \leq 15$。

k	$f(k)$	$s(k)$
2	7.9870	22
3	3.8085	14
4	2.3960	11
5	1.9610	9
6	1.6065	7
7	1.4515	6
8	1.2540	5
9	1.1980	5
10	1.0995	4
11	1.0705	4
12	1.0345	4
13	1.0120	3
14	1.0015	3
15	0.9940	3

每个 $2 \leq k \leq 15$ 对应一个总分为 $s(k)$ 的子任务，每个子任务的得分是子任务中每个测试点的得分的最小值。

每个测试点的得分为所在子任务的总分的 $\max\left(0, 1-\sqrt{\max\left(0, \frac{m}{n \cdot f(k)}-1\right)}\right)$ 倍。

Problem E. 众数

Time Limit 1000 ms
Mem Limit 524288 kB

Description

你有 n 个数对 $(a_1, b_1), \ldots, (a_n, b_n)$。

定义**下标** $i(1 \le i \le n)$ 的权值为将 a_1 个 b_1，a_2 个 b_2，…，a_i 个 b_i 拼接在一起后形成的数组的**最大众数**（特别地，若所有数出现次数相同则为最大数）乘以 a_i。

接下来你有 m 个操作，分两种：

- `1 x y`，将 a_x 增加 y。**保证 y 非负。**
- `2 q`，求最小的正整数 k 使得**下标** $n-k+1 \sim n$ 的权值异或和为 q。

2 操作保证有解，且所有答案之和（记为 $\sum k$）不超过 5×10^7。

Input

输入的第一行有三个正整数 T, n, m，分别表示测试点编号（样例为 0，Hack 数据是 $26 \sim 100$ 之间的自然数）、数对个数和操作个数。

第二行有 $2n$ 个正整数 $a_1, b_1, a_2, b_2, \ldots, a_n, b_n$，其中第 $2i-1$ 和第 $2i$ 个数分别表示第 i 个数对的 a_i, b_i。

之后有 m 行，每行有一个操作，格式见题目描述。

Output

对于每个 2 操作，输出一行一个正整数表示答案。

Sample 1

Input	Output
0 4 5 2 1 3 3 1 1 1 2 2 0 1 4 6 2 6 1 3 8 2 7	2 4 1

Hint

【样例解释】

最开始的四个数组为 $(2,1),(3,3),(1,1),(1,2)$。以计算下标 $2,4$ 的权值为例展示权值的计算方法：

- 要计算下标 2 的权值，就要把 2 个 1、3 个 3 拼在一起得到 $[1,1,3,3,3]$，最大众数为 3；$a_2=3$，所以权值为 $3\times 3=9$。
- 要计算下标 4 的权值，就要把 2 个 1、3 个 3、1 个 1、1 个 2 拼在一起得 $[1,1,3,3,3,1,2]$，最大众数为 3；$a_4=1$，所以权值为 $3\times 1=3$。

以此类推，可知下标 $1,2,3,4$ 的权值依次为 $2,9,3,3$。当 $k=2$ 时，$3,4$ 的权值异或和 $3\oplus 3$ 恰为 0，符合题意。

接下来将 a_4 增加 6 变成 7，显然前 3 个下标权值不变，下标 4 的权值变成 $2\times 7=14$。此时所有下标权值异或和 $2\oplus 9\oplus 3\oplus 14=6$，符合题意，所以此时 $k=4$。

然后把 a_3 增加 8 变成 9。现在 $1\sim 4$ 的权值依次为 $2,9,9,7$，此时 $k=1,3$ 时对应的权值异或和都是 7，此时取更小的 k，所以输出 1。

【数据范围】

记 L 为所有操作结束后，所有 a_i 的最大值（例如样例中，$L=9$。）

测试点编号	$n,m\le$	特殊性质
$1\sim 2$	30	$a_i,y\le 30$
$3\sim 4$	500	
$5\sim 7$	2000	
$8\sim 9$	3×10^5	只有 2 操作
10	3×10^5	$b_i=1$
$11\sim 12$	3×10^5	$b_i\le 2$

测试点编号	$n,m\leq$	特殊性质
$13\sim 15$	3×10^5	$\sum k\leq 5\times 10^5$
$16\sim 19$	3×10^5	$k\leq 300$
$20\sim 25$	3×10^5	

对于全部数据，保证 $3\leq n,m\leq 3\times 10^5$，且 $1\leq b_i\leq n$，$1\leq a_i\leq L\leq 10^9$。

2 操作保证有解，且 $\sum k\leq 5\times 10^7$。

提示：本题读入量较大，建议选用较快的输入输出方式。

Problem F. 编程测试

Time Limit 3000 ms
Mem Limit 1048576 kB

Background

请使用 C++17 或 C++20 提交本题

你需要在程序开头加入如下代码：

```
#include <vector>

std::vector<int> testset(std::vector<int> A, std::vector<int> B, std:
```

题目译自 2022년도 국제정보올림피아드 대표학생 선발고사 - 2차 선발고사 T1「 코딩 테스트 」

注：原题时间限制为 2.5s，但是洛谷评测机不给力，因此放宽至 3s。

Description

公司最近因为编程测试的流行，开始出一些编程测试题目并出售给 IT 公司。

为了方便，公司将题目的难度分为 0 到 $N-1$ 个等级。目前，公司有 A_i 个难度为 i 级的题目，还有 B_i 个难度不确定是 i 级还是 $i+1$ 级的题目。除此之外，没有其他难度的题目。

公司正在寻找愿意购买题目的企业。目前有 M 家企业表示有购买意向，编号从 0 到 $M-1$。第 $j(0 \le j \le M-1)$ 家企业只对难度在 L_j 级到 U_j 级之间的题目感兴趣。

公司计划将难度在 L_j 级到 U_j 级之间的题目按难度各选一个，组成一套题出售给第 j 家企业，我们称之为一场比赛。如果只向第 j 家企业出售题目，最多可以出售多少场比赛？对于难度不确定是 i 级还是 $i+1$ 级的题目，需要适当分配难度，使得出售的比赛数量最多，并且每场比赛中的题目不能重复使用。

你需要实现以下函数：

`vector<int> testset(vector<int> A, vector<int> B, vector<int> L, vector<int> U);`

- 该函数只会被调用一次。
- `A`：长度为 N 的整数数组。对于每个 $i(0 \le i \le N-1)$，A_i 是难度为 i 级的题目数量。

- `B`：长度为 $N-1$ 的整数数组。对于每个 $i(0 \le i \le N-2)$，B_i 是难度不确定是 i 级还是 $i+1$ 级的题目数量。
- `L, U`：长度为 M 的整数数组。对于每个 $j(0 \le j \le M-1)$，L_j, U_j 分别是第 j 家企业感兴趣的题目的最小难度和最大难度。
- 该函数返回一个长度为 M 的整数数组 S。对于每个 $j(0 \le j \le M-1)$，S_j 是可以出售给第 j 家企业的比赛的最大数量。

注意，提交的代码中不应包含任何输入输出操作。

Input

示例评测程序按以下格式读取输入：

- 第 1 行：$N\ M$
- 第 2 行：$A[0]\ A[1]\ \cdots\ A[N-1]$
- 第 3 行：$B[0]\ B[1]\ \cdots\ B[N-2]$
- 第 $4+j(0 \le j \le M-1)$ 行：$L[j]\ U[j]$

Output

示例评测程序的输出格式如下：

- 第 $1+j(0 \le j \le M-1)$ 行：$S[j]$

Sample 1

Input	Output
4 2 2 3 1 1 1 3 2 0 3 1 2	3 5

Hint

样例解释 #1

考虑 $N=4, M=2, A=[2,3,1,1], B=[1,3,2], L=[0,1], U=[3,2]$ 的情况。

评测程序将调用如下函数：

```
testset({2,3,1,1},{1,3,2},{0,1},{3,2});
```

第 0 家企业需要难度在 0 级到 3 级之间的题目。可以通过以下方式组成 3 场比赛，剩下一个难度为 1 级的题目，无法再组成更多场比赛。

	0 级	1 级	2 级	3 级
比赛 1	0	1	2	3
比赛 2	0	1	$1-\mathbf{2}$	$2-\mathbf{3}$
比赛 3	$\mathbf{0}-\mathbf{1}$	$\mathbf{1}-\mathbf{2}$	$1-\mathbf{2}$	$2-\mathbf{3}$

第 1 家企业需要难度在 1 级到 2 级之间的题目。可以通过以下方式组成 5 场比赛，使用了所有可能的题目，无法再组成更多的比赛。

	1 级	2 级
比赛 1	1	2
比赛 2	1	$1-\mathbf{2}$
比赛 3	1	$1-\mathbf{2}$
比赛 4	$0-\mathbf{1}$	$\mathbf{2}-\mathbf{3}$
比赛 5	$\mathbf{1}-\mathbf{2}$	$\mathbf{2}-\mathbf{3}$

因此，调用的 `testset` 函数应返回 $S=[3,5]$。

数据范围

对于所有输入数据，满足：

- $2 \le N \le 10^5$
- $1 \le M \le 10^5$
- 对于所有 $i(0 \le i \le N-1)$，$0 \le A[i] \le 10^8$
- 对于所有 $i(0 \le i \le N-2)$，$0 \le B[i] \le 10^8$
- 对于所有 $i(0 \le j \le M-1)$，$0 \le L[i] \le U[i] \le N-1$

详细子任务附加限制及分值如下表所示。

Subtask	分值	约束
1	3	$A[i] \le 1000 (0 \le i \le N-1), B[i] \le 1000 (0 \le i \le N-2), U[j] - L[j] \le 2 (0 \le j \le M-1)$
2	15	$M \le 100$
3	36	$N \le 5000$
4	23	对于所有 $j (0 \le j \le M-1)$, $L[j] = 0$
5	23	无附加限制

Problem G. 穿越

Time Limit 5000 ms

Mem Limit 1048576 kB

Background

请使用 C++17 或 C++20 提交本题

你需要在程序开头加入如下代码：

```
#include <vector>

void init(int N, int M, std::vector<int> Y1, std::vector<int> Y2);
long long minimize(int A, int B);

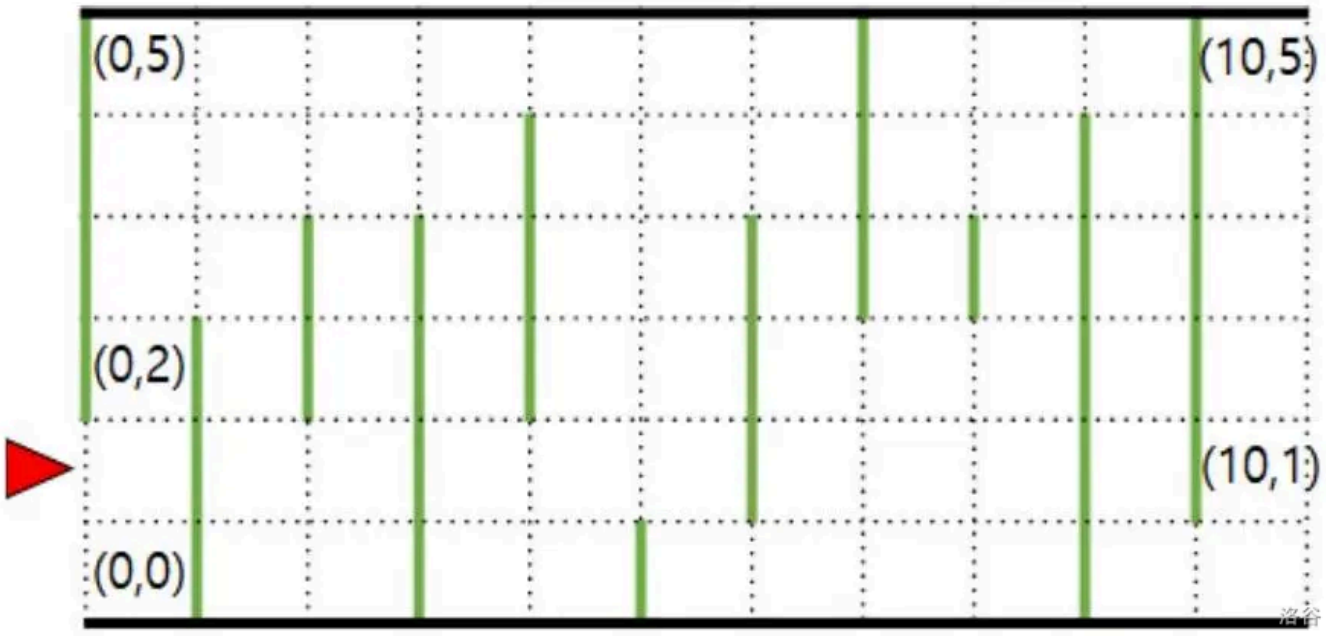
```

题目译自 2020년도 국제정보올림피아드 대표학생 선발고사 – 1차 선발고사 T2「 뚫기」

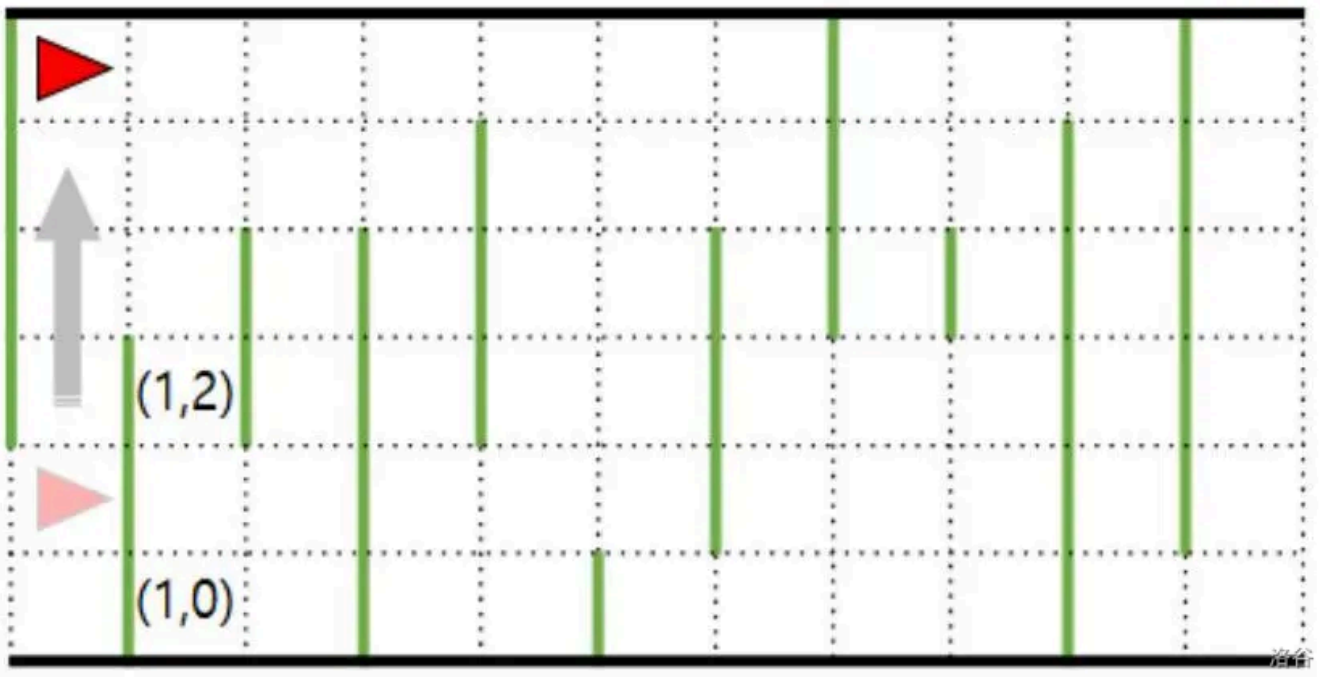
Description

在不久的将来，新加坡流行一种叫做「穿越」的游戏。游戏规则很简单：玩家需要控制一个楔形飞船从左到右穿过一个 $N \times M$ 的隧道。隧道中有 N 个绿色障碍物阻挡飞船前进。障碍物位于每个格子的左侧墙壁上，跨越多个格子的连续障碍物被视为一个障碍物。为了方便描述，飞船前进的方向为 x 轴，与之垂直的方向为 y 轴。在相同的 x 坐标上只能有一个障碍物。

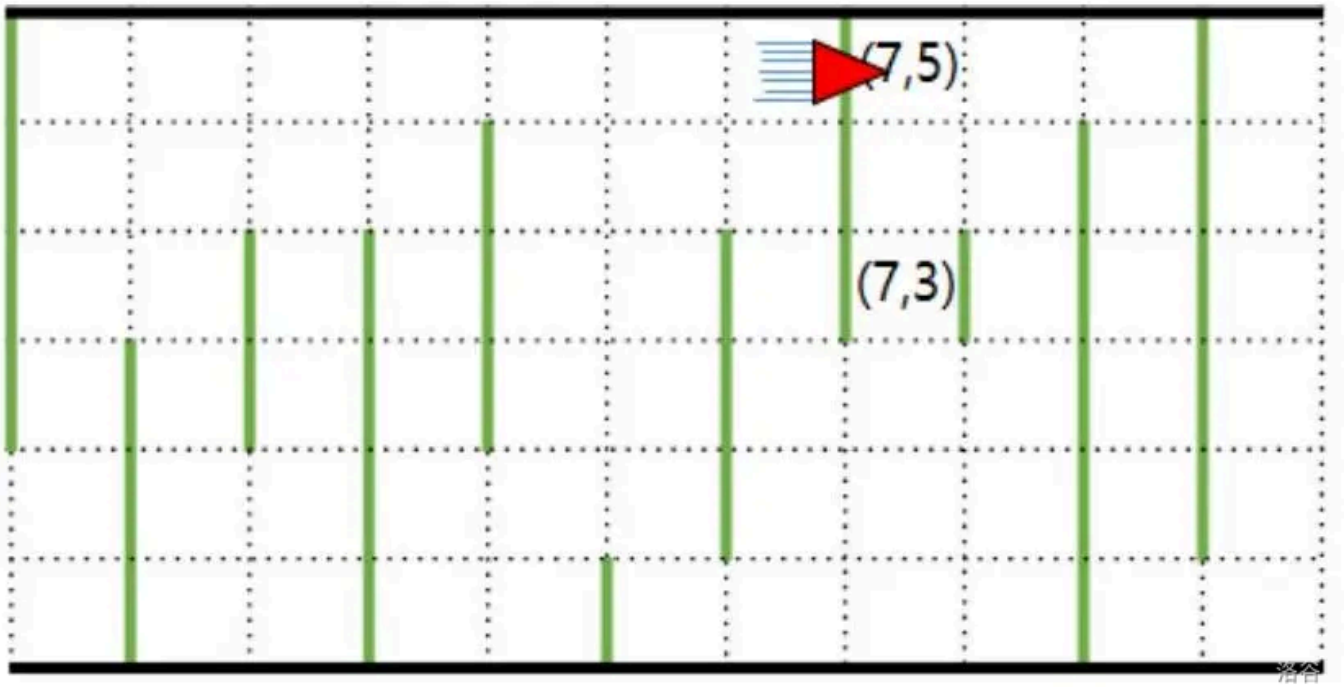
例如，下图展示了一个 11×6 的隧道，其中有 11 个障碍物。最左边的障碍物从 $(0, 2)$ 到 $(0, 5)$，最右边的障碍物从 $(10, 1)$ 到 $(10, 5)$。



飞船在每个格子中可以进行两种移动之一。第一种移动是瞬移。瞬移是将飞船从当前格子移动到相同 x 坐标的任意格子。无论移动到哪个格子，瞬移的费用始终为 A。例如，下图展示了飞船从 $(0,1)$ 瞬移到 $(0,5)$ 的情况，费用为 A。



飞船的第二种移动是前进，费用为 0。即使前进方向上有障碍物，飞船也可以穿过障碍物，但这会产生费用 B。例如，下图展示了飞船从 $(6,5)$ 前进到 $(7,5)$ 的情况，由于 $(7,5)$ 有障碍物，费用为 B。



当飞船完全穿过隧道时，游戏结束。游戏得分是飞船穿过隧道时产生的总费用。显然，得分越低越好。游戏开始时，玩家可以选择飞船的初始 y 坐标，且不产生费用。游戏结束时，飞船的 y 坐标无关紧要。

即使隧道的大小和障碍物的位置相同，费用 A 和 B 的变化也会影响游戏得分和飞船的最佳移动策略。你需要实现以下两个函数，利用给定的隧道大小和障碍物位置，在每次费用 A 和 B 变化时，计算可能的最低游戏得分。

```
void init(int N, int M, int Y1[], int Y2[]);
```

- 该函数在程序开始时调用一次。
- 参数 N 和 M 表示隧道的大小 $N \times M$。
- 参数 $Y1$ 和 $Y2$ 是表示障碍物位置的数组，长度为 N。如果从 $(X, Y_1[X])$ 到 $(X, Y_2[X])$ 有一个障碍物，则 $Y1[X]$ 和 $Y2[X]$ 分别表示 Y_1 和 Y_2。

```
long long minimize(int A, int B);
```

- 参数 A 是飞船瞬移一次的费用，B 是飞船穿过一次障碍物的费用。该函数返回飞船穿过隧道所需的最小费用。

你需要提交一个代码，该代码中实现以下函数：

```
void init(int N, int M, int Y1[], int Y2[]);
long long minimize(int A, int B);
```

这些函数应按照上述描述工作。当然，你可以创建其他函数并在内部使用。提交的代码不应执行输入输出操作或访问其他文件。

Input

示例评测程序按以下格式读取输入：

- 第 1 行：$N\ M\ Q$（$N \times M$：隧道的大小，Q：查询的数量）
- 第 $2+i\ (0 \le i \le N-1)$ 行：$Y_1\ Y_2$（x 坐标为 i 的障碍物的 y 坐标位置从 Y_1 到 Y_2）
- 第 $2+N+i\ (0 \le i \le Q-1)$ 行：$A\ B$（A：瞬移费用，B：穿过障碍物的费用）

Output

示例评测程序对每个查询调用你的代码中的 `minimize()` 函数，并输出返回值。

Sample 1

Input	Output
3 5 2 2 4 0 2 1 3 2 1 3 5	1 3

Hint

样例说明 #1

函数调用	返回值
`init(3, 5, {2, 0, 1}, {4, 2, 3})`	
`minimize(2, 1)`	1
`minimize(3, 5)`	3

数据范围

对于所有输入数据，满足：

- $1 \leq N \leq 10^4$
- $1 \leq M \leq 10^9$
- $1 \leq Q \leq 10^6$
- $0 \leq Y_1 \leq Y_2 \leq M-1$
- $0 \leq A, B \leq 10^9$

详细子任务附加限制及分值如下表所示。

子任务编号	分值	子任务附加限制
1	7	$Q=1, N \leq 3,000, M \leq 3,000$
2	22	$Q \leq 50$
3	19	$N \leq 500$
4	21	$N \leq 2,500$
5	31	无附加限制

Problem F. Vacation

Input file:	standard input
Output file:	standard output
Time limit:	4 seconds
Memory limit:	1024 megabytes

Prof. Pang has an annual leave of c days and he wants to go on vacation.

Now there are n days in a year. Prof. Pang can gain a_i happiness if he rests on the i-th day. The values of happiness, a_i, may be negative.

Prof. Pang wants you to do m operations:

- 1 x y, change the happiness of the x-th day to y.
- 2 l r, Prof. Pang wants to find a period of vacation in $[l, r]$. He wants to rest for several (possibly 0) days in a row and gain as much happiness as possible. However, he only has c days off, thus he can rest for no more than c consecutive days in $[l, r]$.

That means he wants to find

$$\max\left(\max_{\substack{l\le l'\le r'\le r\\ r'-l'+1\le c}}\left(\sum_{i=l'}^{r'} a_i\right), 0\right).$$

Input

The first line contains three integers $n, m, c (1 \le n \le 2 \times 10^5, 1 \le m \le 5 \times 10^5, 1 \le c \le n)$ indicating the number of days in a year, the number of operations, and Prof. Pang's annual leave days.

The next line contains n integers $a_1, a_2, \ldots, a_n (-10^9 \le a_i \le 10^9)$ indicating the values of happiness of every day.

The next m lines are the m operations in the format described above.

It is guaranteed that $1 \le x \le n, -10^9 \le y \le 10^9, 1 \le l \le r \le n$.

Output

For each operation of the second type, print the answer.

Example

standard input	standard output
5 6 3 0 -5 -3 8 -3 2 3 5 1 2 5 2 1 5 1 4 -3 2 3 5 2 1 5	8 10 0 5

Problem I. 略有变化

Time Limit 4000 ms

Mem Limit 1024 kB

下面给你出一道与原题结构相似、但略有变化的题 — 题目风格、输入输出与「极好区间的长度在一个区间内」这一核心思想保持一致，但权值的定义不同，处理技巧也有变化。😊

题目描述

给定一个长度为 n 的整数序列 $a_1, a_2, \ldots, a_n$。

给定一个长度为 n 的整数序列 $b_1, b_2, \ldots, b_n$。

给定两个正整数 C, D（$1 \le C \le D \le n-1$）。

有 q 次询问，其中第 j $(1 \le j \le q)$ 次询问将会给出 L_j, R_j $(1 \le L_j \le R_j \le n)$。定义区间 $[l, r]$ $(1 \le l < r \le n)$ 是**极坏的**，当且仅当区间 $[l, r]$ 的长度在 $[C+1, D+1]$ 内，即 $C \le r - l \le D$。定义区间 $[l, r]$ $(1 \le l < r \le n)$ 的**权值**为 $\dfrac{b_r - a_l}{r - l}$。求出所有被 $[L_j, R_j]$ **包含**的极坏区间的最大权值，即 $\max_{C \le r-l \le D}\{\dfrac{b_r - a_l}{r - l} | L_j \le l < r \le R_j\}$。

输入格式

输入的第一行包含三个正整数 n, C, D，表示序列长度、题目中的限制常数。

输入的第二行包含 n 个整数 $a_1, a_2, \ldots a_n$。

输入的第三行包含 n 个整数 $b_1, b_2, \ldots b_n$。

输入的第四行包含一个正整数 q，表示询问次数。

输入的第 $j+4 (1 \le j \le q)$ 行包含两个整数 L'_j, R'_j，表示第 j 次询问。

本题强制在线，你需要将 L'_j, R'_j 异或上 A' 才能得到真正的 L_j, R_j，即 $L_j = L'_j \oplus A', R_j = R'_j \oplus A'$，其中 $\oplus$ 表示**二进制按位异或**，A' 是上一次查询得到的答案取绝对值后的整数部分，若是第一次查询则 $A' = 0$。不保证 $1 \le L'_j \le R'_j \le n$。保证异或后 $1 \le L_j \le R_j \le n$，且 $[L_j, R_j]$ 中至少有一个极坏区间，即 $R_j - L_j \ge C$。

输出格式

对于每次询问，输出一行一个**既约分数** p/q，表示询问的答案。注意：对于任意**分数** p'/q'，存在**唯一的分数** p/q 满足 $p/q = p'/q'$ 且 $q > 0, \gcd(|p|, q) = 1$，则记 p'/q' 对应的既约分数为 p/q。

样例输入

```
8 2 4
5 3 9 4 1 1 1 9
1 3 10 1 7 2 7 6
9
2 7
0 6
0 9
2 4
2 6
3 4
0 4
4 9
1 11
```

样例输出

```
3/1
-1/1
3/1
3/1
5/2
5/2
4/3
3/1
3/1
```

样例解释

强制在线加密前的样例：

```
8 2 4
5 3 9 4 1 1 1 9
1 3 10 1 7 2 7 6
9
2 7
3 5
1 8
1 7
1 5
```

```
1 6
2 6
5 8
2 8
```

数据范围

本题采用子任务测试，你只有通过了一个子任务中的所有测试点才可以获得该子任务对应的分数。

子任务编号	分值	$n, q \leq$	特殊性质
1	5	500	无
2	5	3000	无
3	10	10^5	a_i 不降
4	5	10^5	$a_i = b_i, C = 1, D = n - 1$
5	15	10^5	$C = 1, D = n - 1$
6	20	10^5	$D = n - 1$
7	20	10^6	$D = n - 1$
8	10	5×10^5	无
9	10	10^6	无

对于所有数据：

$1 \leq n, q \leq 10^6, -10^9 \leq a_i, b_i \leq 10^9, 1 \leq C \leq D \leq n - 1, 1 \leq L_j \leq R_j \leq n, R_j - L_j \geq C$。

时间限制：$4\texttt{s}$

空间限制：$1\texttt{GB}$

Problem J. Souvenirs

Time Limit 3000 ms
Mem Limit 524288 kB

Artsem is on vacation and wants to buy souvenirs for his two teammates. There are n souvenir shops along the street. In i-th shop Artsem can buy one souvenir for a_i dollars, and he cannot buy more than one souvenir in one shop. He doesn't want to introduce envy in his team, so he wants to buy two souvenirs with least possible difference in price.

Artsem has visited the shopping street m times. For some strange reason on the i-th day only shops with numbers from l_i to r_i were operating (weird? yes it is, but have you ever tried to come up with a reasonable legend for a range query problem?). For each visit, Artsem wants to know the minimum possible difference in prices of two different souvenirs he can buy in the opened shops.

In other words, for each Artsem's visit you should find the minimum possible value of $|a_s - a_t|$ where $l_i \le s, t \le r_i, s \ne t$.

Input

The first line contains an integer n ($2 \le n \le 10^5$).

The second line contains n space-separated integers $a_1, ..., a_n$ ($0 \le a_i \le 10^9$).

The third line contains the number of queries m ($1 \le m \le 3 \cdot 10^5$).

Next m lines describe the queries. i-th of these lines contains two space-separated integers l_i and r_i denoting the range of shops working on i-th day ($1 \le l_i < r_i \le n$).

Output

Print the answer to each query in a separate line.

Examples

Input	Output
8 3 1 4 1 5 9 2 6 4 1 8 1 3 4 8 5 7	0 1 1 3

Problem K. rpmtdq

Time Limit 2000 ms
Mem Limit 524288 kB

Description

给定一棵有边权的无根树，需要回答一些询问。

定义 $\texttt{dist(i,j)}$ 代表树上点 i 和点 j 之间的距离。

对于每一组询问，会给出 l, r，你需要输出 $\min(\texttt{dist(i,j)})$ 其中 $l \leq i < j \leq r$。

Input

第一行一个整数 n，表示树的节点个数。

接下来 $n-1$ 行，每行三个整数 x, y, z 表示一条连接 x, y 边权为 z 的树边，输入保证构成一棵树。

之后一行一个数 q，表示询问个数。

之后 q 行，每行两个整数 l, r 表示一组询问，如果对于一组询问，找不到任何二元组 (i, j) 满足 $l \leq i < j \leq r$，则输出 -1。

Output

输出 q 行，每行一个整数，表示这组询问的答案。

Sample 1

Input	Output
5 1 2 5 1 3 3 1 4 4 3 5 2 5 1 1 1 4 2 4 3 4 2 5	-1 3 7 7 2

Hint

Idea：nzhtl1477，Solution：Kubic&ccz181078，Code：Kubic，Data：Kubic

对于100%的数据，满足 $n \leq 2 \times 10^5$，$q \leq 10^6$，$1 \leq z \leq 10^9$。

Problem L. 「Cfz Round 7」**终极 AVX2 硬件指令提速**

Time Limit 5000 ms
Mem Limit 1048576 kB

Description

给定一个长度为 n 的数组 v 和包含 n 条「鱼鱼」的二维平面，第 i 条「鱼鱼」的坐标是 (x_i, y_i)，有权值区间 $[l_i, r_i]$。

有 m 个询问，每次询问给出 A_i, B_i, C_i。定义 $f(k, A_i, B_i, C_i) = 1$ 当且仅当 $A_i x_k + B_i y_k + C_i < 0$，否则 $f(k, A_i, B_i, C_i) = 0$。定义集合 S 为所有满足 $f(k, A_i, B_i, C_i) = 1$ 的 $[l_k, r_k]$ 的并集内的整数组成的集合，求 $\sum_{k \in S} v_k$。

Input

第一行包含一个整数 c，表示该测试点所属的子任务编号。样例满足 $c = 0$。

第二行包含两个整数 n, m。

接下来 n 行，第 i 行包含四个整数 x_i, y_i, l_i, r_i。

接下来一行包含 n 个整数 $v_1, \ldots, v_n$。

接下来 m 行，第 i 行包含三个整数 A_i, B_i, C_i。

Output

对于每个询问，输出一行，包含一个整数表示答案。

Sample 1

Input	Output
0 5 2 2 2 4 4 -3 -3 1 1 -3 -1 3 5 1 -1 3 3 -2 3 1 5 12 3955 8019 1664 9231 2 -2 1 3 2 1	22881 18926

Hint

样例 1 解释

对于第 1 个询问，只有第 3 条「鱼鱼」和第 5 条「鱼鱼」满足条件，它们的权值区间分别为 $[3,5]$ 和 $[1,5]$，因此 $S=\{1,2,3,4,5\}$，答案为 $v_1+v_2+v_3+v_4+v_5=22881$。

对于第 2 个询问，只有第 2 条「鱼鱼」和第 3 条「鱼鱼」满足条件，它们的权值区间分别为 $[1,1]$ 和 $[3,5]$，因此 $S=\{1,3,4,5\}$，答案为 $v_1+v_3+v_4+v_5=18926$。

数据范围

对于所有测试数据，均有：

- $1\le n\le 5\cdot 10^4$，$1\le m\le 5\cdot 10^5$；
- 对于所有 $1\le i\le n$，均有 $-10^6\le x_i,y_i\le 10^6$，$1\le l_i\le r_i\le n$；
- 对于所有 $1\le i\le n$，均有 $1\le v_i\le 10^4$；
- 对于所有 $1\le i\le m$，均有 $-10^3\le A_i,B_i\le 10^3$，${A_i}^2+{B_i}^2>0$，$1\le C_i\le 10^9$；

对于除了 Subtask 2 以外的测试数据，保证 n 条「鱼鱼」的 x,y 坐标分别在某个预设的区间内均匀随机选取；并且，对于第 i 条「鱼鱼」，l_i,r_i 和 x_i,y_i 是分别独立地随机选取的，但 l_i,r_i 的分布没有特殊限制。

本题采用捆绑测试。

- Subtask 1（10 points）：$n,m\le 10^3$。
- Subtask 2（18 points）：对于所有 $1\le i\le n$，保证 $\max(|x_i|,|y_i|)=10^6$。
- Subtask 3（18 points）：对于所有 $1\le i\le m$，保证 $|A_i|=|B_i|=1$。
- Subtask 4（24 points）：$n\le 2\cdot 10^4$，$m\le 2\cdot 10^5$。
- Subtask 5（30 points）：无特殊限制。

Problem M. 骰子 / Kostki

Time Limit 2000 ms
Mem Limit 1048576 kB

Description

n 名玩家使用一枚公平的 k 面骰子（骰子各面编号为 1 到 k，即每次投掷得到 1 到 k 中任意点数的概率均为 $\frac{1}{k}$）进行骰子游戏。初始时，每位玩家的得分均为零。

在单次行动中，得分最少的玩家投掷骰子，并将投掷结果加到自己的得分上。若在某一时刻有多名玩家并列得分最低，则由其中随机一名玩家行动。

当任意一名玩家累计得分达到 m 分或以上时，游戏结束。求行动次数的期望值。

Input

输入的唯一一行包含三个整数 n、k、m（$1 \le n, k, m \le 10^6$），分别表示玩家人数、骰子面数以及获胜所需的分数。

Output

输出一个数，表示行动次数的期望值，对 $M = 10^9 + 7$ 取模后输出。

可以证明，答案可以表示为有理数 p/q 的形式，其中 p 和 q 为整数，且 $q \not\equiv 0 \pmod{M}$。请输出 $p \cdot q^{-1} \bmod M$ 的值。换句话说，输出满足 $0 \le x < M$ 且 $x \cdot q \equiv p \pmod{M}$ 的值 x。

Sample 1

Input	Output
2 4 3	457031255

Hint

样例解释

现有两名玩家，一枚四面骰子，目标分数为 3。第一次投掷时，若玩家投出 3 或 4，游戏立即结束（概率为 $\frac{1}{2}$）。否则，第二名玩家进行投掷。同样地，若其投出 3 或 4，游戏结束（概率同为 $\frac{1}{2}$）。若未结束，则有 $\frac{1}{4}$ 的概率两名玩家均为 1 分（情形 A），$\frac{1}{2}$ 的概率一名玩家为 1 分、另一名为 2 分（情形 B），以及 $\frac{1}{4}$ 的概率两名玩家均为 2 分（情形 C）。

- **情形 A：** 第一名玩家投掷，游戏以 $\frac{3}{4}$ 的概率在第三次行动结束。若未结束，第二名玩家投掷，以 $\frac{3}{4}$ 的概率在第四次行动结束。若仍未结束，则游戏必在第五次行动结束（此时得 2 分的玩家投掷，至少再得 1 分）。
- **情形 B：** 第一名玩家投掷，以 $\frac{3}{4}$ 的概率在第三次行动结束，$\frac{1}{4}$ 的概率在第四次行动结束。
- **情形 C：** 游戏必在第三次行动结束。

综合以上各情形，总期望值为：

$$\frac{1}{2}\cdot 1+\frac{1}{4}\cdot 2+\frac{1}{4}\cdot\left(\frac{1}{4}\cdot\left(\frac{3}{4}\cdot 3+\frac{1}{4}\cdot 4\right)+\frac{1}{4}\cdot\left(\frac{3}{4}\cdot 4+\frac{1}{4}\cdot 5\right)\right)+\frac{1}{2}\cdot\left(\frac{3}{4}\cdot 3+\frac{1}{4}\cdot 4\right)+\frac{1}{4}\cdot 3=\frac{461}{4^4}$$

由于 $256^{-1}=285156252 \bmod M$，且 $461\cdot 285156252\equiv 457031255 \pmod M$，故答案为 457031255。

Problem N. 狼人杀

狼人杀是一款经久不衰的身份推理类桌游。在一局狼人杀的游戏中，每名玩家都会被分配到"村民"、"狼人"、"预言家"等身份。狼人需要尽可能地隐藏自己，预言家可以在晚上查验玩家的身份，而村民则要在预言家的带领下找出狼人。

今天，九条可怜和她的小伙伴们开了一局 n 名玩家参与的游戏。在这局游戏中，玩家从 1 到 n 编号，而可怜的编号是 m。

与传统狼人杀不同的是，这局游戏中的身份包含一名狼人，一名预言家，和 $n-2$ 名村民。每个晚上，只有预言家能够行动（即狼人并不能刀人）：扮演预言家的玩家需要选择一个玩家编号的区间 $[l,r]$，而他/她会得知狼人的编号是否在这个区间中。

可怜分配到的身份是狼人，于是她想要估算一下自己隐藏身份的难度。假设可怜的每名小伙伴都有相同的概率扮演预言家（每名小伙伴 $\frac{1}{n-1}$ 的概率），且每个晚上预言家从所有区间中等概率地选择查验区间（每个区间 $\frac{2}{n(n+1)}$ 的概率）。可怜想要知道期望在多少个夜晚后，预言家得到的信息能够唯一确定可怜就是那名狼人。

Input

输入包含一行两个整数 $n,m(2\le n\le 150,1\le m\le n)$，表示玩家的总数和可怜的编号。

注意：本题的代码长度限制为 8kB。

Output

输出一行一个整数，表示答案对 $1,000,000,007$ 取模后的值，换句话说，如果答案的最简分数表示为 x/y，那么你需要输出 $x\times y^{1,000,000,005}\bmod 1,000,000,007$ 的值。

Examples

Input 1

```
2 2
```

Output 1

```
0
```

Input 2

```
3 2
```

Output 2

```
2
```

Input 3

```
3 3
```

Output 3

```
750000007
```

Input 4

```
10 5
```

Output 4

```
470594541
```

Notes

在第一组数据中，只有两名玩家，因此预言家不需要发动技能就能知道另一名玩家（即可怜）是狼人。

在第二组数据中，不妨假设预言家的编号是1（另一种情况对称），那么只有当预言家问出 $[1,2],[2,2]$ 或者 $[3,3]$ 时，才能唯一确定狼人身份。因此预言家在第 i 个夜晚后才能首次确定可怜身份的概率是 $(1/2)^i$。所以，答案就是 $\sum_{i=1}^{+\infty}(1/2)^i\cdot i=2$。

在第三组数据中，根据预言家编号的不同有着两种情况。

- 如果预言家的编号是 1，那么只有当预言家问出 $[1,2],[2,2]$ 或 $[3,3]$ 时才能唯一确定狼人身份，此时期望需要的夜晚数为 2。
- 如果预言家的编号是 2，那么只有当预言家问出 $[1,1],[1,2],[2,3]$ 或 $[3,3]$ 时才能唯一确定狼人身份，此时期望需要的夜晚数量为 $3/2$。

综合两种情况，答案就是 $(2+(3/2))/2=7/4$。

Scoring

子任务一（23 分），$1\le n\le 20$。

子任务二（34 分），$1\le n\le 50$。

子任务三（43 分），$1\le n\le 150$。

注意：本题的代码长度限制为 8kB。

Problem O. 自然公园 / Natural Park

Time Limit 2000 ms
Mem Limit 262144 kB

Description

JOI 岛是一个观光区，整座岛屿被指定为自然公园。

JOI 岛上有 N 个地点和若干条道路。这些地点从 0 到 $N-1$ 编号。每条道路连接两个不同的地点，且可双向通行。每个地点最多连接 7 条道路。任意两个不同地点之间至多仅有一条道路相连。只要经过若干条道路，我们便可从任意地点到达其他任意地点。

你和你的朋友 IOI 女士将共同调查 JOI 岛。为高效完成调查，你需要弄清 JOI 岛的结构。JOI 岛十分危险，因为岛上栖息着许多野生动物。由于 IOI 女士拥有出色的运动能力，她将负责实地探索 JOI 岛，而你则根据 IOI 女士的报告来确定 JOI 岛的结构。

你将提供两个地点 A、B 以及若干中间地点给 IOI 女士，并询问：若仅允许经过给定的中间地点，是否可能从地点 A 到达地点 B？随后，IOI 女士将探索 JOI 岛，并将结果报告给你。

由于调查时间不能过长，查询次数应小于或等于 45000。

任务

编写一个程序，与 IOI 女士通信并确定 JOI 岛的结构。

实现细节

你需要编写一个程序，实现确定 JOI 岛结构的方法。你的程序应当声明函数 `void Answer(int A, int B);` 和 `int Ask(int A, int B, int Place[]);`，并且使用不低于 C++17 的语言标准提交试题。

你的程序应实现以下函数：

- `void Detect(int T, int N)`

  该函数仅被调用一次。

  - 参数 T 为子任务编号，N 为地点数量。

你的程序应通过调用以下函数，输出其确定的 JOI 岛结构：

- `void Answer(int A, int B)`

  调用该函数的次数应等于 JOI 岛中道路的总数。

- 参数 A、B 表示存在一条连接地点 A 与地点 B 的道路。

参数应满足以下条件：

- A、B 应满足 $0 \leq A < B \leq N-1$。若此条件不满足，你的程序将被判定为 **Wrong Answer[1]**。
- 若函数以参数 (A, B) 被调用，则必须存在一条连接地点 A 与地点 B 的道路。若此条件不满足，你的程序将被判定为 **Wrong Answer[2]**。
- 函数不应以相同的参数 (A, B) 被调用超过一次。若此条件不满足，你的程序将被判定为 **Wrong Answer[3]**。

此外，你的程序可调用以下函数：

- `int Ask(int A, int B, int Place[])`

  该函数用于向 IOI 女士提问。

  - `Place` 是一个数组的指针，表示可能经过的中间地点。对于每个 i（$0 \leq i \leq N-1$），若 `Place[i] = 1`，表示可经过地点 i；若 `Place[i] = 0`，表示不可经过地点 i。
  - 若仅允许经过数组 `Place[]` 中指定的某些地点，可以从地点 A 到达地点 B，则该函数返回值为 1；否则返回值为 0。

  参数应满足以下条件：

  - $0 \leq A < B \leq N-1$。
  - $0 \leq \mathrm{Place}[i] \leq 1$（$0 \leq i \leq N-1$）。
  - $\mathrm{Place}[A] = 1$。
  - $\mathrm{Place}[B] = 1$。

若上述条件未被满足，你的程序将被判定为 **Wrong Answer[4]**。然而，若数组 `Place[]` 的长度不等于 N，该函数的行为无法保证。

函数 `Ask` 的调用次数不得超过 45000 次。若超出，你的程序将被判定为 **Wrong Answer[5]**。

当函数 `Detect` 执行完毕后，若存在某条道路未作为先前对函数 `Answer` 的调用参数出现，则你的程序将被判定为 **Wrong Answer[6]**。

你的程序可实现其他函数供内部使用，或使用全局变量。你的程序不得使用标准输入和标准输出，也不得通过任何方式与其他文件通信。

编译与测试运行

你可以从竞赛网页下载一个归档文件，其中包含一个用于测试你程序的示例评测程序。该归档文件还包含你的程序的一个示例源代码文件。

一个示例评测程序由一个源文件组成，该文件名为 `grader.c` 或 `grader.cpp`。例如，如果你的程序名为 `park.c` 或 `park.cpp`，你可以运行以下命令来编译你的程序。

- C
  `gcc -std=c11 -O2 -o grader grader.c park.c -lm`
- C++
  `g++ -std=c++14 -O2 -o grader grader.cpp park.cpp`

当编译成功后，将生成可执行文件 `grader`。

请注意，实际的评测程序与示例评测程序不同。示例评测程序将以单个进程运行，从标准输入读取输入数据，并将结果写入标准输出。

Input

示例评测程序从标准输入读取以下数据：

- 第一行输入包含一个整数 T，表示子任务编号。
- 第二行输入包含一个整数 N，表示地点数量。
- 第三行输入包含一个整数 M，表示道路数量。
- 在接下来的 M 行中，第 i 行（$1 \leq i \leq M$）包含两个以空格分隔的整数 A_i、B_i。这表示存在一条连接地点 A_i 与地点 B_i 的道路，且该道路可双向通行。

Output

当程序成功终止时，示例评测程序将向标准输出写入以下信息。（引号实际不会输出。）

- 若你的程序被视为正确，示例评测程序将输出“Accepted.”。
- 若你的程序被视为 **Wrong Answer**，示例评测程序将按以下格式输出其类型：“Wrong Answer [1]”，随后你的程序将被终止。

若你的程序被视为多种类型的 **Wrong Answer**，示例评测程序仅报告其中一种。

Sample 1

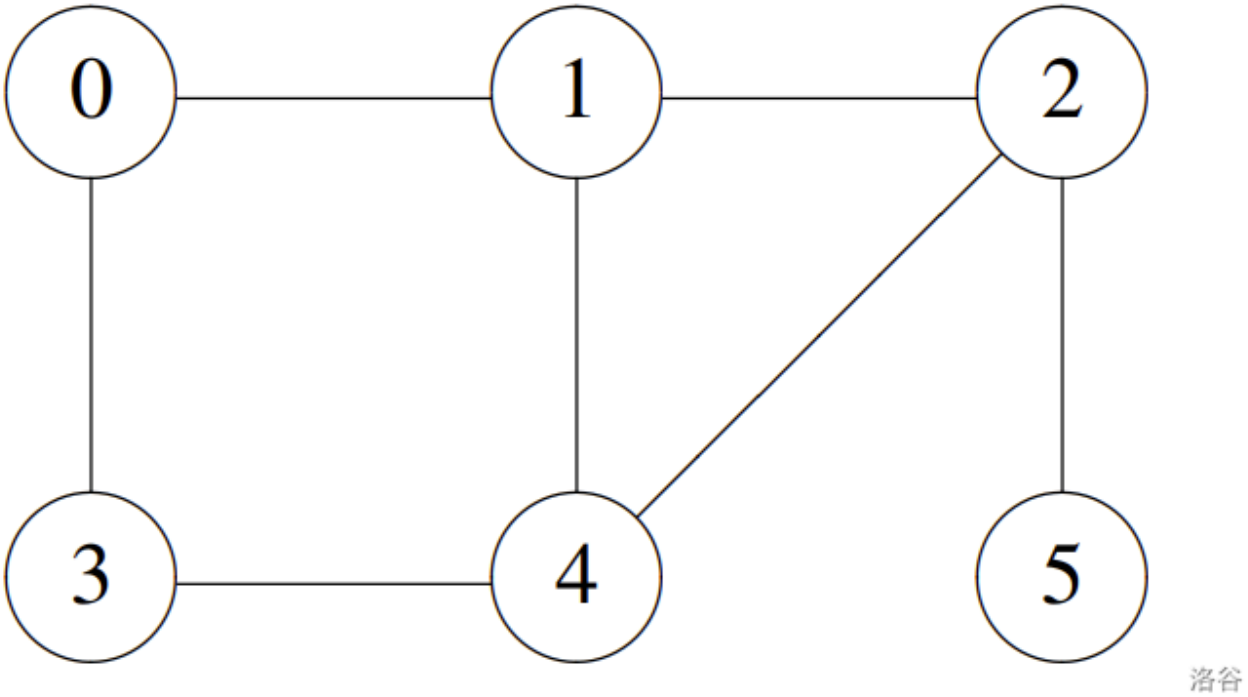
Input	Output
1 6 7 0 1 0 3 1 2 1 4 2 4 2 5 3 4	

Hint

示例调用

调用	返回值
`Ask(3, 5, {0,0,1,1,1,1})`	1
`Answer(2, 4)`	
`Answer(2, 5)`	
`Answer(3, 4)`	
`Ask(0, 4, {1,0,1,0,1,0})`	0
`Answer(0, 1)`	
`Answer(0, 3)`	
`Answer(1, 4)`	
`Answer(1, 2)`	

请注意，本示例中的函数调用不一定具有实际意义。在本示例中，函数 `Detect` 以参数 $T = 1$、$N = 6$ 被调用。在本示例中，JOI 岛屿的结构如下：



JOI 岛屿的结构。
圆圈和数字表示地点及其编号，线段表示道路。

- 第一次调用函数 `Ask` 是询问：若仅允许经过地点 2、3、4、5，是否可以从地点 3 到达地点 5。由于可行，函数 `Ask` 返回 1。
- 第二次调用函数 `Ask` 是询问：若仅允许经过地点 0、2、4，是否可以从地点 0 到达地点 4。由于不可行，函数 `Ask` 返回 0。

约束条件

所有输入数据均满足以下条件。关于 T、N、M 的含义，请参见“示例评测程序的输入”。

- $1 \leq T \leq 5$。
- $2 \leq N \leq 1400$。
- $1 \leq M \leq 1500$。
- 对于每个地点，最多有 7 条道路连接它与其他地点。
- 若允许经过若干条道路，则可以从任意地点到达其他任意地点。
- 对于任意两个不同的地点，最多只有一条道路连接它们。

子任务

共有 5 个子任务。每个子任务的得分及附加约束如下：

子任务 1 [10 分]

- $T = 1$。

- $N \leq 250$。

子任务 2 [10 分]

- $T = 2$。
- $M = N - 1$。
- 对于地点 0 或 $N - 1$，恰好有一条道路连接它与其他地点；对于其他每个地点，恰好有两条道路连接它与其他地点。

子任务 3 [27 分]

- $T = 3$。
- $M = N - 1$。
- 对于每个 i（$1 \leq i \leq N - 1$），若我们最多经过 8 个其他地点，则可以从地点 0 到达地点 i。

子任务 4 [30 分]

- $T = 4$。
- $M = N - 1$。

子任务 5 [23 分]

- $T = 5$。

翻译由 Qwen3-235B 完成

Problem D. Solar Panel Grid Optimization

In the not-so-distant future, the world is managed by advanced robots capable of performing complex tasks to maintain global systems. One such system is responsible for managing a massive grid of solar panels that generate renewable energy for an entire city. These panels are arranged in an $N \times N$ matrix, where each cell in the matrix can either be in an active (1) or inactive (0) state.



Figure 1: By AleSpa - Own work, CC BY-SA 3.0

To optimize energy production, the robots need to transform the current state matrix A of the solar panel grid into a desired state matrix B. The transformation process involves two types of operations:

- Shift an entire row to the left, i.e., select a positive integer i between 1 and N and transform $(A_{i,1}, A_{i,2}, \ldots, A_{i,N})$ into $(A_{i,2}, A_{i,3}, \ldots, A_{i,N}, A_{i,1})$.
- Shift an entire column downwards, i.e., select a positive integer i between 1 and N and transform $(A_{1,i}, A_{2,i}, \ldots, A_{N,i})$ into $(A_{N,i}, A_{1,i}, A_{2,i}, \ldots, A_{N-1,i})$, and then flip the state of the top cell, making the new $A_{1,i} \leftarrow 1 - A_{1,i}$.

You are tasked with finding a sequence of operations that can transform the grid from its current state A to the desired state B by using no more than 10^3 operations.

Input

The first line of the input contains a single integer N ($3 \le N \le 20$).

The next N lines describe the initial matrix A. The j-th character on the i-th line of these lines will be either 0 or 1, representing the value of $A_{i,j}$.

After these N lines, there will be an empty line.

The next N lines describe the target matrix B. The j-th character on the i-th line of these lines will be either 0 or 1, representing the value of $B_{i,j}$.

Output

The first line of the output should be an integer T ($0 \le T \le 10^3$), representing the number of the operations you used.

The next T lines of the output describe the operations. The i-th line of these lines should be one of the following:

- `row` i: Perform an operation on the i-th row.
- `column` i: Perform an operation on the i-th column.

If there are multiple possible solutions, you may print any of them.

Example

standard input	standard output
4 1011 1100 0100 1011 1001 0110 0110 1001	4 row 2 row 2 row 2 column 3

Problem Q. The Riddle of the Sphinx

Time Limit 1000 ms
Mem Limit 262144 kB

What walks on four feet in the morning, two in the afternoon, and three at night?

This is an interactive problem. This problem doesn't support hacks.

Sphinx's duty is to guard the city of Thebes by making sure that no unworthy traveler crosses its gates. Only the ones who answer her riddle timely and correctly (or get an acc for short) are allowed to pass. As of those who fail, no one heard of them ever again...

So you don't have a choice but to solve the riddle. Sphinx has an array $a_1, a_2, \ldots, a_n$ of **nonnegative** integers **strictly smaller** than 2^b and asked you to find the maximum value among its elements. Of course, she will not show you the array, but she will give you n and b. As it is impossible to answer this riddle blindly, you can ask her some questions. For given i, y, she'll answer you whether a_i is **bigger** than y. As sphinxes are not very patient, you can ask at most $3 \cdot (n + b)$ such questions.

Although cunning, sphinxes are honest. Even though the array **can change** between your queries, answers to the previously asked questions will remain valid.

Input

The first line contains two integers n and b ($1 \le n, b \le 200$). The remaining parts of the input will be given throughout the interaction process.

Interaction

In each round your program must output a single line with an integer i ($0 \le i \le n$) and a binary string of length **exactly** b denoting the **binary representation** of y (most significant bit first).

If $i > 0$, this line encodes the question: *Is a_i bigger than y?*. There should be at most $3 \cdot (n + b)$ such lines; after each of them, the interactor will print `yes` or `no` in a single line.

If $i = 0$, this is the last round of interaction, after which your program should terminate, and y should be the maximal value among the elements of Sphinx's array. Note that this round **does not** count to the query limit.

Note that the interactor **is adaptive**.

After printing a query, do not forget to output the end of the line and flush the output. Otherwise, you will get `Idleness limit exceeded`. To do this, use:

- `fflush(stdout)` or `cout.flush()` in C++;
- `System.out.flush()` in Java;
- `flush(output)` in Pascal;
- `stdout.flush()` in Python;
- see the documentation for other languages.

If your solution does not correctly follow the interaction guideline above, it may receive an arbitrary verdict. Otherwise, your program will receive the `Wrong Answer` judgment if it reports the wrong maximum.

Examples

Input	Output
5 3 yes no no no no yes	5 101 5 110 4 100 3 101 2 001 1 000 0 110

Input	Output
4 3 no no no no	1 000 2 000 3 000 4 000 0 000

Input	Output
1 1	0 0

Note

In all examples, the sequence is **fixed** beforehand.

In the first example, the sequence is $2, 1, 4, 0, 6$.

In the second example, the sequence is $0, 0, 0, 0$.

In the third example, the sequence is 0.

Note that if the interactor was adaptive, then the interaction in the first and the third example would not be sufficient to return the correct value of maximum.

Problem R. Research

Time Limit 3000 ms
Mem Limit 524288 kB

Research: Draw cards from 1 of the facedown Level Decks equal to your Research Amount. You may choose 1 to Build or File.

Alice and Bob will play a game with a deck of cards. This deck consists of $n - 1$ white cards and one green card. Initially, the green card is located at the s-th place from the top of the deck, with the number 1 written on it. Both Alice and Bob know the place of the green card.

Alice goes first. In each turn, they should proceed as follows:

- Take k cards from the top of the deck. (If the deck has fewer than k cards, take all of them.) After that, discard 0 or 1 card among those k cards, and put the remaining cards at the bottom of the deck in any order they like.
- If the green card is discarded by Bob, the game ends immediately with a score of 0.
- If the green card is discarded by Alice, the game ends immediately with a score equal to the number written on the green card.
- If Alice takes the green card and returns it to the deck, she should tell Bob the new location of the green card and increase the number on the green card by 1.

Alice needs to maximize the score, while Bob needs to minimize it. You have to find the final score of the game if both of them play optimally.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 50$). The description of the test cases follows.

The only line of each test case contains three integers n, k, and s ($1 \le n, k \le 10^9$, $1 \le s \le n$).

Output

For each test case, if the game will never end, output a line with the word `Infinity`; otherwise, output a single integer — the score of the game.

Examples

Input	Output
6 3 2 1 5 3 4 10 3 1 7 3 7 817247666 7237 327476688 610723117 332458760 292738094	2 0 2 4 3470 278264358

Note

In the first test case, an optimal process of the game is as follows. Alice should take 2 cards from the top, discard nothing, and return the green card to the bottom of the deck. Then Bob should take 2 white cards from the top and discard one. On the third turn, Alice should discard the green card with the number 2 written on it.

In the second test case, Bob will take the green card first and discard it. Therefore, the score should be 0.